LangChain, LangGraph and Multi-Agent Systems

Prepared By

SREEDEV C S

November 6, 2026

Contents

1 Introduction **1**

2 LangChain **2**
2.1 What is LangChain? . . . 2
2.2 Core Components of LangChain . . . 2
2.2.1 Language Models . . . 2
2.2.2 Prompts . . . 2
2.2.3 Chains . . . 2
2.2.4 Tools . . . 3
2.2.5 Retrievers . . . 3
2.2.6 Agents . . . 3

3 LangGraph **4**
3.1 What is LangGraph? . . . 4
3.2 Fundamental Building Blocks . . . 4
3.2.1 State . . . 4
3.2.2 Nodes . . . 4
3.2.3 Edges . . . 4
3.2.4 Conditional Edges . . . 5
3.2.5 Persistence and Checkpointing . . . 5
3.3 Conditional Workflows and Cycles . . . 5

4 LangChain vs LangGraph **6**

5 Agent Lifecycle **8**
5.1 Introduction to AI Agents . . . 8
5.2 Runtime Lifecycle of an AI Agent . . . 8
5.2.1 Perception and Input Processing . . . 8
5.2.2 Planning and Reasoning . . . 8
5.2.3 Decision Making . . . 8
5.2.4 Action and Execution . . . 9
5.2.5 Observation and Evaluation . . . 9
5.2.6 Completion or Iteration . . . 9

5.3 Agent Development and Operational Lifecycle 9

6 Multi-Agent Coordination Patterns (MCP) 11
6.1 Introduction to Multi-Agent Systems 11
6.2 Need for Multi-Agent Coordination 11
6.3 Comparison of Multi-Agent Coordination Patterns 12

7 Practical Implementation 13
7.1 Example: Multi-Agent Research Assistant 13
7.2 Role of LangChain and LangGraph 14
7.3 Implementing Multi-Agent Coordination Using LangGraph 14

8 Challenges and Best Practices 15
8.1 Challenges 15
8.2 Best Practices 15

Chapter 1

Introduction

Large Language Models (LLMs) have significantly changed the way intelligent applications are developed. Initially, LLM applications primarily relied on simple prompt-response interactions. However, as application requirements became more complex, the need emerged for systems capable of performing multiple operations, interacting with external tools, maintaining context, and making decisions autonomously.

Frameworks such as LangChain and LangGraph provide development abstractions for building such applications. LangChain simplifies the integration of language models, prompts, tools, and retrieval systems, whereas LangGraph provides graph-based orchestration for stateful and long-running agent workflows [1, 2].

An AI agent can be understood as a system that uses a language model to determine the actions required to accomplish a particular objective. Unlike a conventional LLM application that follows a fixed sequence of operations, an agent can dynamically select tools, evaluate intermediate results, and decide its next action.

As tasks become increasingly complex, multiple specialized agents can collaborate to solve different parts of a problem. Such systems are known as multi-agent systems. Their effectiveness depends on how agents communicate, delegate tasks, share information, and coordinate execution.

Chapter 2

LangChain

2.1 What is LangChain?

LangChain is an open-source framework designed to simplify the development of applications powered by Large Language Models. It provides reusable components and integrations that enable developers to connect language models with external tools, databases, APIs, and retrieval systems [1]. The framework supports the development of applications ranging from simple question-answering systems to tool-using autonomous agents.

2.2 Core Components of LangChain

2.2.1 Language Models

Language models form the reasoning and generation component of an LLM application. They process the input provided by the application and generate responses based on the given context.

2.2.2 Prompts

Prompts define the instructions and contextual information provided to a language model. Prompt templates allow developers to dynamically construct prompts using variables and user-provided information.

2.2.3 Chains

Chains represent sequences of operations in which the output of one component can serve as the input to another. They enable developers to organize multiple processing steps into a structured workflow.

2.2.4 Tools

Tools enable an LLM-powered application to interact with external systems. Examples include web searches, calculators, database queries, and API calls.

2.2.5 Retrievers

Retrievers identify and return relevant information from external knowledge sources. They are commonly used in Retrieval-Augmented Generation (RAG) applications.

2.2.6 Agents

Agents use language models to determine which actions should be performed to accomplish a task. They can dynamically select and invoke tools based on the current context and intermediate results.

Chapter 3

LangGraph

3.1 What is LangGraph?

LangGraph is a framework for building stateful, multi-step, and long-running agent workflows. It represents application execution as a graph consisting of nodes, edges, and shared state. Unlike a strictly linear workflow, LangGraph supports conditional transitions, loops, parallel execution, persistence, and human-in-the-loop interactions [2].

3.2 Fundamental Building Blocks

3.2.1 State

State represents the shared data maintained throughout the execution of a graph. It stores information such as user inputs, intermediate results, messages, and execution status.

3.2.2 Nodes

Nodes are individual processing units within a graph. A node can execute an LLM call, invoke a tool, perform data processing, or execute any other application logic. Each node receives the current state and can return updates to it.

3.2.3 Edges

Edges define the execution flow between nodes. They determine which node should execute after the current node completes.

3.2.4 Conditional Edges

Conditional edges enable dynamic routing based on the current state or the output of a decision-making function.

3.2.5 Persistence and Checkpointing

LangGraph supports persistence through checkpointers, which save graph execution state at specific stages. This enables applications to resume interrupted workflows, preserve conversation context, and implement human-in-the-loop operations.

3.3 Conditional Workflows and Cycles

LangGraph supports workflows in which the next execution step is determined dynamically. Conditional edges allow the graph to select different execution paths depending on the current state. Cycles are particularly useful in agentic applications. For example, an agent may repeatedly invoke a language model, execute a selected tool, observe the result, and return to the reasoning stage until the task is completed. This graph-based execution model provides greater flexibility for implementing autonomous agents and complex workflows.

Chapter 4

LangChain vs LangGraph

Although LangChain and LangGraph are closely related, they address different aspects of LLM application development. LangChain provides higher-level abstractions for integrating language models, prompts, tools, and retrieval components. LangGraph focuses on explicit workflow orchestration, state management, conditional execution, and persistence. The frameworks can be used together, with LangChain providing agent components and LangGraph controlling their execution flow [1, 2].

Aspect	**LangChain**	**LangGraph**
Primary Purpose	Framework for developing LLM-powered applications and agents.	Framework for orchestrating stateful and complex agent workflows.
Architecture	Provides high-level abstractions and reusable components.	Uses a graph-based architecture consisting of nodes, edges, and shared state.
Execution Model	Supports chains and agent-based execution.	Supports graph-based execution with explicit control over workflow transitions.
Workflow Structure	Suitable for sequential processing and standard agent workflows.	Supports sequential, branching, cyclic, and parallel workflows.
State Management	Supports agent state and conversation history.	Provides explicit shared state that can be updated throughout graph execution.
Control Flow	Provides abstractions for model and tool interactions.	Provides explicit control through normal and conditional edges.
Tool Integration	Provides integrations for tools, APIs, databases, and external services.	Can orchestrate tool execution through individual graph nodes and agents.
Persistence	Supports persistence through compatible integrations and agent runtimes.	Provides checkpoint-based persistence for saving and resuming execution.
Human-in-the-Loop	Can support human interaction through agent integrations.	Supports interrupt-based workflows for human approval and intervention.
Best Suited For	Building LLM applications, RAG pipelines, and tool-using agents.	Building complex, stateful, and long-running agentic workflows.
Relationship	Can be used to develop individual agents and application components.	Can orchestrate LangChain agents and other application components.

Table 4.1: Comparison between LangChain and LangGraph

Chapter 5

Agent Lifecycle

5.1 Introduction to AI Agents

An AI agent is an intelligent system that can perceive information, reason about a given objective, select appropriate actions, and execute those actions to accomplish a task. Unlike traditional applications that follow a completely predefined execution sequence, an autonomous agent can dynamically determine its next action based on the available information and the results of previous actions.

5.2 Runtime Lifecycle of an AI Agent

The runtime lifecycle describes the sequence of operations performed by an agent when it receives a task.

5.2.1 Perception and Input Processing

The agent receives an input from the user or an external system. It interprets the request and identifies relevant contextual information.

5.2.2 Planning and Reasoning

The agent analyzes the objective and determines the steps required to accomplish it. Complex tasks may be decomposed into smaller subtasks.

5.2.3 Decision Making

Based on the current context and execution plan, the agent determines the most appropriate action. This may involve selecting a tool, querying

a database, or delegating a task to another agent.

5.2.4 Action and Execution

The selected action is executed. The agent may interact with external APIs, databases, search engines, or other software systems through tools.

5.2.5 Observation and Evaluation

After execution, the agent observes the result and evaluates whether the action successfully contributed to the objective.

5.2.6 Completion or Iteration

If the objective has been achieved, the agent generates the final response. Otherwise, it uses the observed results to revise its plan and continue execution.

This iterative process enables agents to adapt their behavior based on intermediate outcomes.

5.3 Agent Development and Operational Lifecycle

Apart from its runtime execution, an AI agent also undergoes a broader development and operational lifecycle. This lifecycle ensures that agents are designed, evaluated, deployed, and maintained systematically.

1. **Planning:** Define the problem, objectives, expected behavior, and success criteria.
2. **Design:** Select the language model, tools, data sources, memory architecture, and orchestration strategy.
3. **Development:** Implement the agent logic, prompts, tool integrations, and workflows.
4. **Testing and Evaluation:** Evaluate correctness, reliability, security, and performance.

5. **Deployment:** Integrate the agent into the target application or production environment.

6. **Monitoring:** Track execution, latency, token consumption, errors, and overall performance.

7. **Optimization and Maintenance:** Improve the agent through updated prompts, models, tools, and workflows.

8. **Retirement:** Decommission agents that are no longer required and revoke their access to external resources.

A systematic lifecycle is important because an agent's behavior can change when its model, prompts, tools, or external dependencies are modified [3].

Chapter 6

Multi-Agent Coordination Patterns (MCP)

6.1 Introduction to Multi-Agent Systems

A multi-agent system consists of multiple autonomous agents that collaborate to accomplish a common objective. Each agent is assigned a particular role or responsibility and may possess specialized knowledge, tools, or capabilities. Instead of assigning an entire complex task to a single agent, the system decomposes the problem into smaller subtasks and distributes them among specialized agents. The agents coordinate their activities by exchanging messages, sharing information, delegating tasks, and combining their individual outputs [4].

6.2 Need for Multi-Agent Coordination

Multi-agent systems are useful when a problem requires multiple specialized capabilities, parallel processing, or independent evaluation.

Some important advantages include:

- **Specialization:** Agents can focus on specific domains or tasks.
- **Task Decomposition:** Complex problems can be divided into manageable subtasks.
- **Parallel Execution:** Independent tasks can be executed simultaneously.

- **Modularity:** Individual agents can be developed, tested, and modified independently.
- **Scalability:** Additional agents can be introduced when new capabilities are required.

The major coordination patterns are:

- **Supervisor Pattern:** It follows a centralized coordination architecture in which a central agent controls the execution of multiple specialized agents.
- **Sequential or Pipeline Pattern:** Here multiple agents execute their tasks in a predefined order.
- **Parallel Coordination Pattern:** It allows multiple agents to execute independent subtasks simultaneously.
- **Hierarchical Coordination Pattern:** It organizes agents into multiple levels. A high-level agent decomposes the overall objective and delegates responsibilities to intermediate agents.
- **Handoff Pattern:** It allows an agent to transfer control of a task to another agent when a particular condition or requirement is satisfied.
- **Peer-to-Peer or Network Pattern:** Here, the agents communicate directly with other agents to exchange information, request assistance, and coordinate their activities.

6.3 Comparison of Multi-Agent Coordination Patterns

Pattern	**Coordination**	**Suitable Use Case**
Supervisor	Centralized delegation	Complex task management
Sequential	Fixed execution order	Data processing pipelines
Parallel	Concurrent execution	Independent analysis tasks
Hierarchical	Multi-level delegation	Large-scale systems
Handoff	Dynamic transfer of control	Customer support
Peer-to-Peer	Direct agent communication	Distributed collaboration

Table 6.1: Comparison of Multi-Agent Coordination Patterns

Chapter 7

Practical Implementation

7.1 Example: Multi-Agent Research Assistant

Consider an AI-powered research assistant designed to generate comprehensive reports on user-provided topics. The system consists of multiple specialized agents, each responsible for a particular stage of the report-generation process.

- **Planner Agent:** Decomposes the user's request into smaller subtasks.
- **Research Agent:** Collects relevant information from available sources.
- **Analysis Agent:** Evaluates and interprets the collected information.
- **Verification Agent:** Checks the reliability and consistency of the findings.
- **Writer Agent:** Generates the final report using the processed information.
- **Supervisor Agent:** Coordinates execution and manages task delegation.

The overall architecture can be implemented using LangChain for agent construction and LangGraph for workflow orchestration.

7.2 Role of LangChain and LangGraph

LangChain can be used to define the individual agents, their language models, prompts, and tools. LangGraph can then coordinate these agents using nodes, edges, shared state, and conditional routing. This combination enables developers to build modular, stateful, and controllable multi-agent applications.

7.3 Implementing Multi-Agent Coordination Using LangGraph

LangGraph provides a graph-based execution model that can be used to coordinate multiple specialized agents. Each agent can be represented as a node in the graph, while edges determine the sequence or conditions under which control is transferred between agents. The shared state maintains information exchanged during execution. Conditional edges enable dynamic routing, allowing the system to select an appropriate agent based on the current task requirements. Furthermore, individual agent workflows can be organized as subgraphs, enabling modular system design and reuse. Thus, LangGraph provides the orchestration capabilities required to implement various multi-agent coordination patterns.

Chapter 8

Challenges and Best Practices

8.1 Challenges

Despite their flexibility, LangChain, LangGraph, and multi-agent systems introduce several implementation challenges such as:

- **Context Management:** Maintaining relevant contextual information across multiple agents is chal- lenging. Excessive context sharing can increase token consumption, while insufficient context may result in incomplete or inconsistent outputs.
- **Communication Overhead:** Communication between multiple agents introduces additional latency and computational overhead.
- **Error Propagation:** An incorrect output generated by one agent may affect subsequent agents. Validation mechanisms should be incorporated at important stages.
- **Execution Loops:** Agentic workflows may enter unnecessary execution loops if termination conditions are not properly defined.
- **Cost and Performance:** Multiple agents may require additional model inference calls, increasing token consumption and execution time.

8.2 Best Practices

- Use the simplest architecture that satisfies the application requirements.

- Define clear roles and responsibilities for every agent.
- Restrict access to tools and external resources.
- Implement error handling and retry mechanisms.
- Define explicit termination conditions.
- Monitor execution, latency, and token usage.
- Evaluate the complete workflow rather than individual agent responses alone.

Bibliography

[1] https://docs.langchain.com/oss/python/langchain.

[2] https://docs.langchain.com/oss/python/langgraph.

[3] https://www.ibm.com/think/topics/agent-lifecycle-management.

[4] https://aws.amazon.com/marketplace/build-learn/ai-agent-learning-series/multi-agent-architectures.